Iterative LLM-Based NAS with Feedback Memory

Xiaojie Gu, Dmitry Ignatov, Radu Timofte
Computer Vision Lab, CAIDAS & IFI, University of Würzburg, Germany

Abstract

Neural Architecture Search (NAS) automates network design, but early methods typically demanded substantial computational resources. We propose a closed-loop pipeline that leverages large language models (LLMs) to iteratively generate, evaluate, and refine convolutional neural network architectures for image classification. Central to our approach is a historical feedback memory *mechanism inspired by Markov chains: a sliding window of the most recent improvement attempts is fed back to the LLM, enabling it to learn from past successes and failures without additional training. Using one-epoch proxy accuracy on CIFAR-10 [17], CIFAR-100 [17], and ImageNette [11] as a fast architecture ranking signal, we evaluate the pipeline with three LLMs across up to 2000 iterations. On CIFAR-10, DeepSeek-Coder-6.7B-Instruct improves generated architectures from 28.2% to a peak of 69.2%, and other models have also achieved varying degrees of improvement on other datasets. Compared to prior iterative LLM optimization methods that rely on global elite retention (discarding failure trajectories) [26, 34], require complex population maintenance [3], or operate in constrained discrete search spaces [39], our bounded Markovian memory explicitly models sequential code execution failures in an* unconstrained open code space. *This structured causal learning from recent mistakes is crucial for hardware-constrained environments, enabling the generation of suitably compact models. These results establish that LLMs of varied origins and specializations can serve as effective architecture designers when equipped with iterative feedback and historical memory. Crucially, a full 2000-iteration search completes in approximately 18 GPU hours on a single consumer-grade GPU (RTX 4090), offering a highly data-efficient and low-budget paradigm for LLM-driven NAS.*

1. Introduction

Neural Architecture Search (NAS) has emerged as a powerful paradigm for automating the design of deep neural networks, achieving competitive or superior performance to hand-crafted architectures across diverse tasks [20, 25, 40]. However, early conventional NAS methods are notoriously resource-intensive: early reinforcement learning approaches required up to 22,400 GPU-days [40], evolutionary algorithms demanded thousands of GPU-days [25], and even efficient differentiable relaxations [20] depend on heavy supernet training. This severe computational cost has motivated data-efficient and training-free approaches that use proxy metrics to rank architectures without full training [18, 21], yet these methods still operate within constrained, predefined search spaces (e.g., cell-based structures).

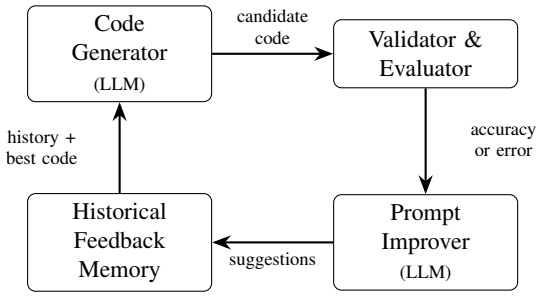
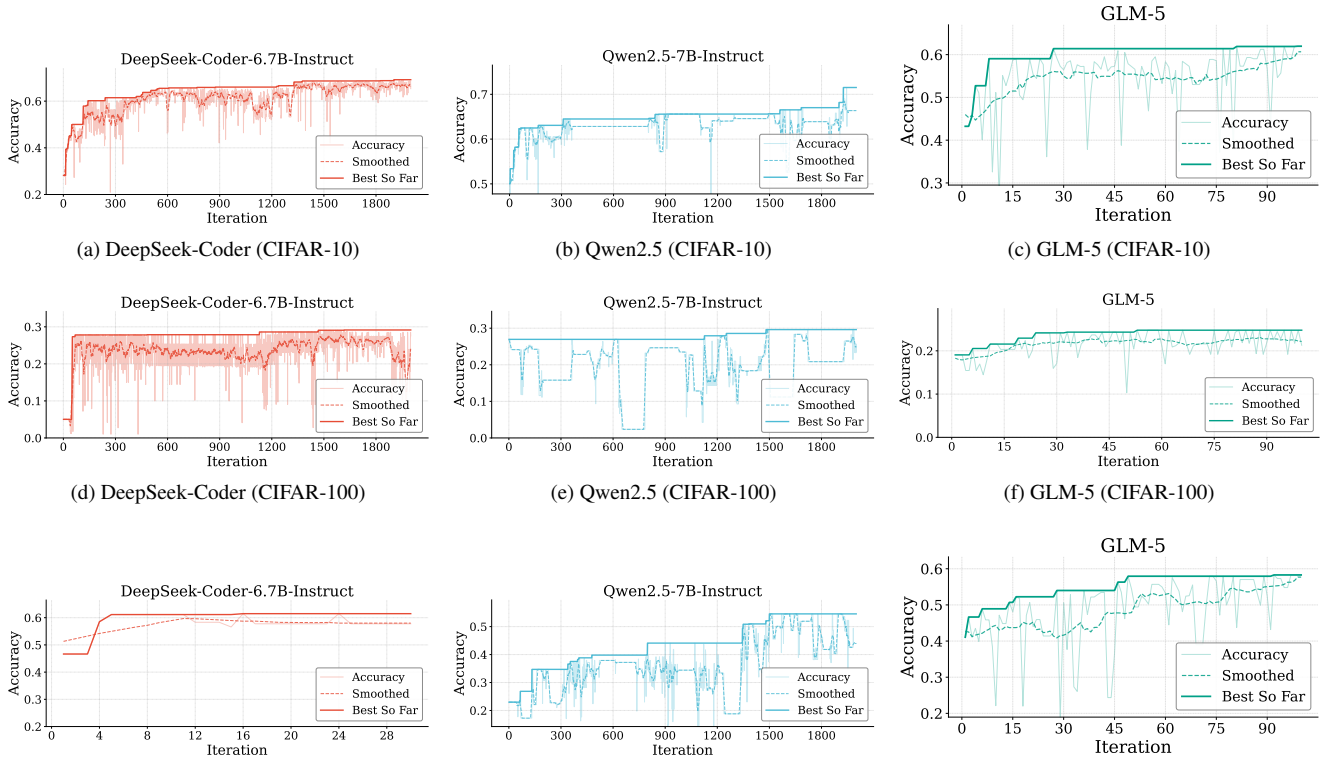


Figure 1. Overview of the iterative NAS pipeline. The Code Generator produces a candidate architecture as executable PyTorch code. The Evaluator validates and trains it using one-epoch proxy evaluation. The Prompt Improver analyzes results with historical feedback memory to generate targeted improvement suggestions for the next iteration.

Recent advances in large language models (LLMs) have opened a fundamentally different avenue: using LLMs as architecture generators that produce executable neural network code directly [14, 16, 32]. Unlike traditional NAS, LLM-based approaches operate in the *unconstrained open code space* of object-oriented programs rather than fixed cell-based encodings, enabling more flexible and expressive architectural invention. Prior work within the NNGPT framework [16] and related efforts [12, 23, 30] have demonstrated that LLMs can generate functional vision models, but predominantly through single-shot or few-shot generation without iterative self-improvement.

A critical limitation of single-shot LLM generation is



(a) DeepSeek-Coder (CIFAR-10) (b) Qwen2.5 (CIFAR-10) (c) GLM-5 (CIFAR-10)

(d) DeepSeek-Coder (CIFAR-100) (e) Qwen2.5 (CIFAR-100) (f) GLM-5 (CIFAR-100)

(g) DeepSeek-Coder (ImageNette) (h) Qwen2.5 (ImageNette) (i) GLM-5 (ImageNette)

Figure 2. One-epoch proxy accuracy on **CIFAR-10** (top row, a–c), **CIFAR-100** (middle row, d–f), and **ImageNette** (bottom row, g–i) across all iterations. Light curves show per-iteration accuracy (the accuracy of iterations with errors fall back to previous value), dashed lines show the smoothed trend (window w=15), and bold lines show the best-so-far trajectory. All models exhibit clear upward trends. For DeepSeek-Coder on ImageNette, only the first 30 iterations are plotted because all subsequent iterations resulted in errors.

that it treats architecture design as a one-pass prediction, discarding the evaluation signal entirely. In contrast, human engineers iterate: they build, test, analyze failures, and refine their designs based on accumulated experience. This observation motivates our central question: *Can a small LLM iteratively improve neural network architectures by learning from a structured history of its own attempts?*

We propose a closed-loop pipeline comprising three components: (1) a **Code Generator** that prompts an instruction-tuned LLM to produce PyTorch model implementations, (2) an **Evaluator** that trains each generated model for a single epoch on CIFAR-10, CIFAR-100, or ImageNette as a fast proxy for architecture quality, and (3) a **Prompt Improver** that analyzes evaluation results alongside a sliding window of recent iteration history—a mechanism we term *historical feedback memory*—to produce targeted improvement suggestions for the next iteration. This design draws on the Markov property: the improvement decision at each step depends on the current best architecture and a bounded window of recent transitions, rather than the full trajectory.

Our main contributions are:

- A closed-loop, iterative NAS pipeline driven by LLMs that progressively discovers better architectures through code generation, evaluation, and prompt refinement.
- A *historical feedback memory* mechanism that maintains a sliding window of past improvement attempts, enabling the LLM to mitigate repeating failed strategies and build on successful ones.
- An empirical demonstration across three LLMs of different origins and specializations showing that the pipeline consistently improves architecture quality—from 28.2% to 69.2% (DeepSeek-Coder-6.7B-Instruct), 50.0% to 71.5% (Qwen2.5-7B), and 43.2% to 62.0% (GLM-5) on CIFAR-10—requiring only ∼18 GPU hours on a single consumer-grade 24GB GPU, establishing a low-budget approach to NAS.

2. Related Work

Neural Architecture Search. NAS methods aim to automate the design of neural network architectures. Early approaches employed reinforcement learning [40] and evolutionary algorithms [25] to explore architecture search spaces, but required thousands of GPU hours per search.

Parameter-sharing methods such as ENAS [24] and differentiable approaches like DARTS [20] substantially reduced search cost by amortizing evaluation across architectures. More recently, *zero-shot* or *training-free* NAS methods [18, 21] bypass training entirely by scoring architectures at initialization using proxy metrics such as activation overlap [21] or gradient-based indicators. While effective within their respective search spaces, these methods remain constrained to predefined, discrete architecture parameterizations (e.g., cell-based structures) and cannot generate truly novel architectural patterns.

LLMs for Code Generation and AutoML. Large language models have demonstrated strong capabilities in code generation [4], and their application to machine learning automation is an active research area. The NNGPT framework [16] uses LLMs as neural network generators, showing that language models can produce functional vision models from textual prompts. Subsequent work explored fractal-inspired architectures [23], few-shot prompting strategies [32], novel architecture creativity [14], and non-standard channel priors [30]. LLMs have also been applied to hyperparameter optimization [15], data transformation design [29], and collaborative vision-language pipelines [9, 13, 28]. These efforts are supported by curated datasets of neural network architectures [10, 31] and deployment pipelines [6]. However, most existing approaches treat LLM-based architecture generation as a single-step process, without leveraging iterative refinement from evaluation feedback. Our work extends this line by introducing a closed-loop pipeline with historical memory that enables progressive architecture improvement.

Iterative LLM Optimization. Recent work has explored using LLMs as iterative optimizers that progressively improve solutions by learning from historical attempts. OPRO [34] demonstrated that LLMs can optimize text prompts and discrete solutions by conditioning on sorted history of past candidates and scores. FunSearch [26] extended this to program synthesis for mathematical discovery using island-based evolutionary search. EvoPrompting [3] applied evolutionary prompting with LLMs as mutation operators for code-level neural architecture search. ReEvo [35] introduced dual-layer reflective memory for LLM-driven heuristic algorithm design. More directly related to NAS, LLMO [39] used LLMs as combinatorial optimizers within the predefined cell-based search space of NAS-Bench-201 [7], replacing traditional meta-heuristics. However, existing methods face key limitations: reliance on global elite histories that discard crucial failure trajectories [26, 34], the computational overhead of maintaining evolutionary populations [3], operation in constrained discrete search spaces [39], or targeting domains without structural code constraints [35]. Our work introduces bounded Markovian memory specifically designed for iterative neural architecture code generation in open search spaces, with explicit failure modeling enabling effectiveness on small frozen LLMs.

Positioning relative to prior work. Table 1 positions our method relative to the five most similar LLM-based optimization approaches. We are the only method that simultaneously satisfies all five properties: (1) targeting neural architecture search, (2) operating in an unconstrained code search space rather than predefined cells, (3) automated failure diagnosis via structured diagnostic triples, (4) demonstrated effectiveness on a single consumer GPU with $\leq$7B LLMs, and (5) requiring no LLM fine-tuning whatsoever. Notably, while EvoPrompting also performs code-level NAS on small models, it requires soft prompt-tuning of the LLM, whereas our pipeline operates on fully frozen instruction-tuned models. LLMO also applies LLMs to NAS without training, but operates in predefined discrete search spaces that fundamentally limit architectural expressiveness.

3. Methodology

Inspired by recent advancements in the application of LLMs across various domains [9, 13, 15, 28] and prior architectural synthesis experiments within the NNGPT framework [12, 14, 15, 23, 29, 30, 32], and leveraging the existing LEMUR dataset of a broad range of high-capacity and edge-optimized models [6, 10, 31], we developed an iterative NAS pipeline. The pipeline consists of three core modules operating in a closed loop, as illustrated in Figure 1.

3.1. Pipeline Overview

At each iteration t, the pipeline executes the following cycle (see Algorithm 1): (1) the Code Generator produces a candidate architecture $\mathcal{A}_t$ as executable PyTorch code; (2) the validated generated code is trained and evaluated by the Evaluator to obtain a proxy accuracy a_t; (3) the Prompt Improver analyzes the result in the context of the current best architecture and recent iteration history, producing improvement suggestions s_t for the next iteration. The best-performing architecture $\mathcal{A}^* = \arg\max_{i \leq t} \mathcal{A}_i$ is maintained as the reference implementation throughout the search.

3.2. Code Generator

The Code Generator uses a pre-trained, instruction-tuned LLM to produce a complete PyTorch model class implementing `nn.Module`. The LLM receives a fixed prompt template comprising: (i) a role description positioning the model as a "visionary deep learning architect," (ii) the task specification (CIFAR-10, CIFAR-100 or ImageNette im-

Method	NAS target	Unconstrained search space	Automated failure diagnosis	Single-GPU ($\leq$24 GB)	No LLM fine-tuning
OPRO [34]	×	×	×	×	✓
FunSearch [26]	×	✓	×	×	✓
EvoPrompting [3]	✓	✓	×	✓	×
ReEvo [35]	×	✓	✓	×	✓
LLMO [39]	✓	×	×	×	✓
Ours	✓	✓	✓	✓	✓

Table 1. Systematic comparison with most similar LLM-based NAS and optimization methods. NAS target: method aims at neural architecture search; Unconstrained search space: search over executable code rather than predefined cell encodings or fixed discrete spaces; Automated failure diagnosis: structured modeling of code execution failures (e.g., diagnostic triples) as first-class feedback signals; Single-GPU ($\leq$24 GB): demonstrated effective with $\leq$7B LLMs on a single consumer GPU; No LLM fine-tuning: operates with frozen pretrained LLMs without any parameter updates. Our method uniquely combines all five dimensions.

age classification without pre-trained weights), (iii) the current best implementation $\mathcal{A}^*$ as reference code, and (iv) the improvement suggestions s_{t-1} from the previous iteration. The generated code must define a `Net(nn.Module)` class with standard `__init__` and `forward` methods. Generation uses temperature $\tau = 0.7$ and nucleus sampling ($p = 0.9$) to balance architectural diversity with code coherence.

3.3. Evaluator

Generated code undergoes a two-stage evaluation process.

Quick validation. The model is instantiated and a forward pass is performed with a dummy input tensor of shape corresponding to the dataset (e.g., $2\times3\times32\times32$ for CIFAR-10). The output shape is verified to match $B \times C$ (for batch size B and C classes). Models failing this check are discarded with an error message forwarded to the Prompt Improver.

Proxy training. Models passing validation are trained for **one epoch** on the respective dataset (CIFAR-10, CIFAR-100 or ImageNette) using SGD (momentum 0.9, weight decay 5×10^{-4}), an initial learning rate of 0.01 with cosine annealing, and a batch size of 128. Standard data augmentation is applied: random crop with padding and random horizontal flip. Top-1 test accuracy serves as the proxy metric. This one-epoch accuracy provides a fast, informative signal for ranking architectures [27], enabling rapid iteration without the cost of full convergence training.

3.4. Prompt Improver with Historical Feedback Memory

The Prompt Improver is the central component enabling iterative learning. After each evaluation, it receives three inputs: (1) the best code $\mathcal{A}^*$ and its accuracy a^*, (2) the current iteration's code $\mathcal{A}_t$ and its evaluation outcome (accuracy value or error message), and (3) a *historical feedback memory* $\mathcal{H}_t$ containing the last K=5 improvement attempts.

Each history entry $h_i \in \mathcal{H}_t$ is a triple recording: the identified problem found in the previous iteration about the generated code, the suggested improvement, and the resulting outcome (accuracy achieved or error encountered). This sliding-window design draws on the Markov property: the improvement decision at step t depends on the current state (best architecture plus recent history window) rather than the complete trajectory. The bounded window prevents context overflow while providing sufficient signal for the LLM to identify failure patterns.

Markov Property Formalization. Let $\mathcal{A}_t$ denote the architecture generated at step t, a_t its evaluation outcome (accuracy or error), and s_t the improvement suggestions. Our historical feedback memory $\mathcal{H}_t^{(K)}$ maintains exactly the most recent K=5 improvement attempts:

$$\mathcal{H}_t^{(K)} = \{(s_{t-K}, a_{t-K}), \ldots, (s_{t-1}, a_{t-1})\}. \tag{1}$$

The improvement suggestion generation satisfies the **K-order Markov property**:

$$P(s_t \mid \mathcal{A}^*, \mathcal{H}_t) = P(s_t \mid \mathcal{A}^*, \mathcal{H}_t^{(K)}), \tag{2}$$

meaning the next suggestion depends only on the current best architecture and the bounded recent history, not the complete trajectory. This design ensures **constant context size** and avoids the context overflow problem faced by unbounded history approaches like OPRO [34].

Each history entry (s_i, a_i) is structured as a **diagnostic triple**:

$$s_i = (\text{problem}_i, \text{suggestion}_i, \text{outcome}_i), \tag{3}$$

where problem_i identifies the architectural deficiency, suggestion_i proposes concrete code modifications, and outcome_i records whether the suggestion succeeded (accuracy gain) or failed (error type). This structured format enables the LLM to learn causal patterns between design decisions and outcomes, unlike scalar-score histories in prior work [26, 34].

Algorithm 1: Iterative LLM-Driven NAS Pipeline

Input: LLM $\mathcal{L}$, max iterations T, history window K

$\mathcal{A}^* \leftarrow \emptyset$; $a^* \leftarrow 0$; $\mathcal{H} \leftarrow \emptyset$; $s_0 \leftarrow \emptyset$;
for $t \leftarrow 1$ **to** T **do**
  $\mathcal{A}_t \leftarrow \text{Generate}(\mathcal{L}, \mathcal{A}^*, s_{t-1})$;
  if *Validate*$(\mathcal{A}_t)$ *succeeds* **then**
    $a_t \leftarrow \text{TrainAndEvaluate}(\mathcal{A}_t)$;
  else
    $a_t \leftarrow$ error message;
  if $a_t > a^*$ **then**
    $\mathcal{A}^* \leftarrow \mathcal{A}_t$; $a^* \leftarrow a_t$;
  Append (s_{t-1}, a_t) to $\mathcal{H}$; keep last K entries;
  $s_t \leftarrow \text{Improve}(\mathcal{L}, \mathcal{A}^*, \mathcal{A}_t, a_t, \mathcal{H})$;
return $\mathcal{A}^*$, a^*

The Prompt Improver produces a structured response containing: (a) a **reason** diagnosing the current result, (b) an **inspiration** drawing on cross-disciplinary insights, and (c) concrete **improvement suggestions** for the next iteration. These suggestions, together with the best code and the code of current iteration, form the input to the Code Generator in the subsequent iteration, closing the feedback loop.

4. Experiments

4.1. Experimental Setup

Dataset. We evaluate our approach on three datasets: CIFAR-10 [17], CIFAR-100 [17], and ImageNette [11]. CIFAR-100 shares the same 32×32 image dimensions as CIFAR-10 but increases the classification difficulty from 10 to 100 fine-grained classes. Standard augmentation (random crop with padding, random horizontal flip, normalization) is applied during training, inspired by [2].

Language models. To evaluate the pipeline across model specializations, we run separate experiments with three LLMs: code-specialized DeepSeek-Coder-6.7B-Instruct, small generalist Qwen2.5-7B-Instruct, and large generalist GLM-5. All models use the same generation parameters: temperature 0.7, nucleus sampling $p = 0.9$, maximum 2,048 new tokens. A fixed seed with incremental call counter ensures reproducible but diverse generation across iterations. DeepSeek-Coder-6.7B and Qwen2.5-7B are each run for 2000 iterations; GLM-5 is only run for 100 iterations for economic reasons. Additionally, the prompt format utilized for GLM-5 differs from the others by incorporating the implementation codes of the top-K successful architectures, alongside the codes and corresponding evaluation data from the previous K models.

Training protocol. Each candidate architecture is trained for one epoch with SGD (momentum 0.9, weight decay 5×10^{-4}), learning rate 0.01 with cosine annealing, and batch size 128. A fixed random seed (43) and deterministic CUDA operations ensure reproducibility. Training runs in isolated subprocesses with a 30-minute timeout. Fine-tuning of LLMs and training of computer vision models are performed on a NVIDIA GeForce RTX 4090 24G GPU.

Baseline. We compare against **single-shot generation**: the accuracy of the first successfully generated and evaluated architecture (iteration 1), representing what the LLM produces from its pre-trained knowledge alone without any feedback signal.

4.2. Evaluation Metrics

To evaluate our approach rigorously, we adopt true validation accuracy after the first training epoch as our primary architecture quality metric [8], rather than relying on zero-shot NAS proxies. Although zero-shot proxies are correlated with fully trained accuracy [5, 19, 22], they remain indirect indicators of performance. Even the strongest reported Spearman rank-correlation coefficients ($\rho \approx 0.5$–0.82) on standard benchmarks such as NAS-Bench-101 [37] and NAS-Bench-201 [7] correspond to a coefficient of determination of at most $R^2 \approx 0.67$, leaving substantial variance unexplained [1, 33, 36, 38]. By using first-epoch validation accuracy instead, we aim to demonstrate more directly that our algorithm can reliably influence and accelerate early-stage performance trajectories of neural networks.

To characterize search dynamics, we compute: Spearman rank correlation ρ and Kendall τ between iteration index and accuracy. All trends are evaluated for statistical significance.

4.3. Main Results

Table 2 compares the key results across the LLMs. The models exhibit statistically upward trends in architecture quality across all three datasets (Spearman ρ up to 0.75, $p \approx 0$), confirming that the iterative pipeline with historical feedback memory consistently drives improvement.

DeepSeek-Coder-6.7B-Instruct. Among the three models, the code-specialized DeepSeek-Coder stands out for its remarkable reliability on CIFAR-class datasets: it completed 1519 of 2000 iterations successfully on CIFAR-10 (76.0%) and an even higher 1902 of 2000 on CIFAR-100 (95.1%), suggesting that the structured feedback loop becomes more effective as the model accumulates experience with similar low-resolution inputs. This reliability translates into strong improvement trajectories—on CIFAR-10, one-epoch proxy accuracy climbed from a 28.2% baseline to 69.2% ($\rho = 0.754$), the largest absolute gain observed

across all model–dataset combinations. On CIFAR-100, it progressed from 5.0% to 29.2% ($\rho = 0.205$, $p \approx 0$), with a notably higher success rate than on CIFAR-10 despite the increased class count. However, this consistency collapsed entirely on ImageNette: the higher input resolution (160×160) triggered persistent context retention failures, leaving only 13 successful evaluations out of 2000 (0.7%). Even so, the few successful architectures reached 61.5% accuracy from an initial 46.7%, indicating that the model's code quality remained high when generation succeeded.

Qwen2.5-7B-Instruct. At first glance, Qwen2.5-7B appears to be the weakest performer: its CIFAR-10 success rate of just 18.8% (376 of 2000 iterations) is by far the lowest among the three models. Yet this generalist model ultimately achieved the highest peak accuracy on CIFAR-10—71.5%, surpassing both DeepSeek-Coder (69.2%) and GLM-5 (62.0%). This apparent paradox reflects a distinctive exploration strategy: over thousands of iterations, Qwen2.5 increasingly generates ambitious architectures, many of which fail but occasionally yield superior designs. The Spearman correlation remains significant ($\rho = 0.561$, $p \approx 0$), confirming genuine improvement despite the noise. A similar pattern emerges on CIFAR-100, where it maintained a moderate 53.2% success rate (1064 of 2000) and improved from 27.0% to 29.6%, though the weaker correlation ($\rho = 0.037$) suggests that extended error loops on this harder task limited effective search diversity. On ImageNette, the success rate improved to 32.1% (642 evaluations), and proxy accuracy rose substantially from 23.0% to 54.6% ($\rho = 0.663$)—the strongest correlation among all model–dataset pairs for this benchmark.

GLM-5. Running for only 100 iterations, the large generalist GLM-5 from ZhipuAI provides evidence that extensive iteration counts are not strictly necessary for large language models. It achieved the highest success rates across all three datasets—91.0% on CIFAR-10, 85.0% on CIFAR-100, and 81.0% on ImageNette—reflecting a conservative generation strategy that favors reliable, well-structured architectures. On CIFAR-10, proxy accuracy rose from 43.2% to 62.0% ($\rho = 0.422$), with improvement continuing throughout the search and the peak reached at the 91st successful evaluation. The same convergence pattern appeared on CIFAR-100 (19.1% $\rightarrow$ 24.8%, $\rho = 0.431$) and ImageNette (41.1% $\rightarrow$ 58.3%, $\rho = 0.631$). While GLM-5's peak accuracies are below those of models given 20$\times$ more iterations, its high success rate and consistent improvement make it the most reliable search agent in our evaluation.

Figure 2 shows the accuracy trajectories for all models across all three datasets. All curves display consistent upward trends: DeepSeek-Coder achieves the largest absolute improvement on CIFAR-10, Qwen2.5 reaches the highest peak accuracy, and GLM-5 exhibits a robust upward trend across all three datasets. The pipeline generalizes well across datasets of varying difficulty, confirming its broad applicability.

5. Ablation Study

Additionally, we conducted ablation experiments (Figure 3) involving DeepSeek-Coder-6.7B on CIFAR-10, CIFAR-100, and ImageNette to evaluate how removing historical feedback or reference architectures impacts the search trajectory. The results show that without the historical feedback memory or the reference architecture, the search process stagnates or degrades significantly and fails to surpass the single-shot baseline. The anomalous accuracy spike ($\sim$66%) in the CIFAR-10 *No Feedback* trajectory is an incidental discovery that is swiftly lost without history retention. This underscores that explicitly modeling causality from past code execution failures is critical for iterative architectural improvement.

6. Discussion

Effectiveness of iterative refinement. All models exhibit statistically significant positive correlations between evaluation index and accuracy (ρ up to 0.75, Kendall τ up to 0.55, all $p \approx 0$), confirming that historical feedback memory enables systematic architectural improvement across different LLM origins and specializations. Access to the last K=5 improvement attempts provides the LLM with sufficient context to identify recurring failure patterns and avoid previously unsuccessful strategies. All accuracy curves display characteristic increasing behavior—rapid early gains followed by gradual flattening—mirroring patterns in traditional optimization and suggesting that the LLM-driven search exhibits meaningful optimization dynamics rather than random exploration.

Effect of model specialization and iteration scale. The three models reveal distinct behavior along two axes. DeepSeek-Coder-6.7B achieves the largest absolute improvement on CIFAR-10 (+41.0 pp from single-shot to peak) with a high and stable success rate (76.0%) over 2000 iterations, suggesting that code specialization yields both reliable generation and systematic progression in specific situations. Qwen2.5-7B achieves the highest peak accuracy (71.5%) at the cost of a dramatically lower success rate (18.8%) over 2000 iterations—its Markovian exploration increasingly targets complex, ambitious architectures that more often fail validation but, when successful, surpass DeepSeek-Coder's best. GLM-5 demonstrates consistent improvement (+18.7 pp from 43.2% to 62.0%) with

(a) CIFAR-10

Metric	DeepS.	Qwen	GLM
Total iters	2000	2000	100
Succ. evals	1519	376	91
Succ. rate	76.0%	18.8%	91.0%
1st acc.	28.2%	50.0%	43.2%
Best acc.	69.2%	71.5%	62.0%
Improve.	+41.0%	+21.5%	+18.7%
ρ	0.754	0.561	0.422
τ	0.551	0.407	0.300

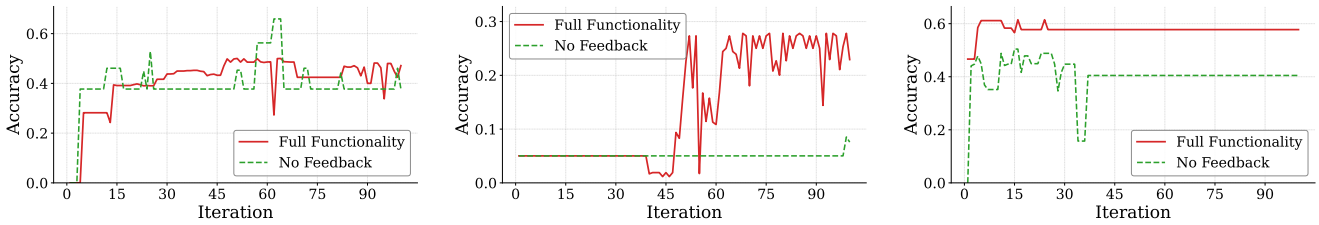
(b) CIFAR-100

Metric	DeepS.	Qwen	GLM
Total iters	2000	2000	100
Succ. evals	1902	1064	85
Succ. rate	95.1%	53.2%	85.0%
1st acc.	5.0%	27.0%	19.1%
Best acc.	29.2%	29.6%	24.8%
Improve.	+24.1%	+2.7%	+5.7%
ρ	0.205	0.037	0.431
τ	0.142	0.031	0.311

(c) ImageNette

Metric	DeepS.	Qwen	GLM
Total iters	2000	2000	100
Succ. evals	13	642	81
Succ. rate	0.7%	32.1%	81.0%
1st acc.	46.7%	23.0%	41.1%
Best acc.	61.5%	54.6%	58.3%
Improve.	+14.8%	+31.6%	+17.2%
ρ	−0.084	0.663	0.631
τ	−0.069	0.444	0.452

Table 2. Comparison of the iterative NAS pipeline across three LLMs on CIFAR-10, CIFAR-100, and ImageNette datasets. The models exhibit statistically significant improvement over single-shot generation. On ImageNette, correlation metrics for DeepSeek are not meaningful due to insufficient successful evaluations. On CIFAR-100, Qwen2.5 shows weaker correlation due to extended error loops reducing effective search diversity.



(a) CIFAR-10 (b) CIFAR-100 (c) ImageNette

Figure 3. Ablation study of DeepSeek-Coder-6.7B-Instruct on CIFAR-10, CIFAR-100, and ImageNette datasets. The results highlight the effectiveness of the complete iterative loop with historical feedback memory compared to its ablated variants.

the highest success rate (91.0%) within only 100 iterations, making it the most reliable search agent of the three.

Comparison with iterative LLM optimization methods. Our approach shares conceptual similarities with recent work on using LLMs as iterative optimizers [26, 34, 35], but differs in two critical dimensions that enable its effectiveness on neural architecture search:

Structured failure modeling. Our diagnostic triples explicitly encode code execution failures as first-class entries in the history, enabling the LLM to learn from structural mistakes. This is fundamentally different from prior work where low-performing solutions are simply discarded from the prompt [26, 34] or managed implicitly through population selection [3]. Failure rates of 5–99% across our runs highlight that code generation for neural architectures is inherently error-prone; ignoring these failures would discard critical learning signals.

Hardware-aware and low-budget search. Unlike early traditional NAS methods that required thousands of GPU days [25, 40], our entire 2000-iteration pipeline completes in approximately 18 GPU hours on a single consumer-grade RTX 4090 GPU. Our results demonstrate that $\leq$7B frozen LLMs of diverse origins can effectively perform NAS in severely resource-constrained environments, whereas prior iterative LLM optimization work primarily relies on massive proprietary models (e.g., GPT-4 and PaLM 2-L in OPRO [34]). Furthermore, by explicitly evaluating in a shared environment where the LLM occupies most of the VRAM, the search implicitly discovers hardware-efficient models tailored to limited memory capacities. We hypothesize this is partly enabled by our dual-LLM specialization: the Code Generator focuses solely on code synthesis given structured suggestions, while the Prompt Improver handles diagnostic reasoning. This task decomposition reduces per-call cognitive load compared to end-to-end generation [3].

One-epoch proxy as architecture ranking signal. Using one-epoch accuracy as a ranking signal offers a pragmatic trade-off between evaluation cost and informativeness. While the absolute accuracy values are well below state-of-the-art results with full training, prior work on training-free NAS [18, 21] has established that cheap proxy metrics can effectively rank architectures. Our one-epoch proxy serves a similar role: it distinguishes between architectures of varying quality at minimal cost, enabling the feedback loop to guide the LLM toward better designs. Retraining the best-discovered architectures with full training schedules could verify whether the one-epoch ranking transfers to converged performance.

Limitations. Our study has several limitations. First, the code generation success rates vary substantially (0.7–95.1%), and the causes of model-specific failure rates at scale (e.g., Qwen2.5's 18.8% success rate over 2000 iterations on CIFAR-10) merit further investigation into prompt design and constrained decoding strategies. Second, while our evaluation now spans three datasets (CIFAR-10, CIFAR-100, and ImageNette), experiments on larger-scale benchmarks and diverse tasks would better demonstrate versatility.

Future work. Promising directions include: (1) retraining the top-K discovered architectures with full training schedules to verify their converged performance; (2) extending the cross-model comparison to additional LLM families and sizes; (3) extending to larger datasets (e.g., ImageNet) and tasks beyond image classification; and (4) apply genetic algorithm for historical feedback memory.

7. Conclusion

We presented an iterative NAS pipeline that leverages instruction-tuned LLMs to progressively generate and improve neural network architectures through a closed loop of code generation, evaluation, and prompt refinement with historical feedback memory. Evaluated on CIFAR-10, CIFAR-100, and ImageNette with three instruction-tuned LLMs across up to 2000 iterations, the pipeline significantly improved one-epoch proxy accuracy. On CIFAR-10, performance improved from 28.2% to a peak of 69.2% for DeepSeek-Coder-6.7B ($\rho = 0.75$), from 50.0% to 71.5% for Qwen2.5-7B ($\rho = 0.56$), and from 43.2% to 62.0% for GLM-5 ($\rho = 0.42$). On CIFAR-100, DeepSeek-Coder achieved the largest improvement from 5.0% to 29.2%, while GLM-5 showed a strong trend ($\rho = 0.43$) despite only 100 iterations. ImageNette results confirmed these positive trends (e.g., Qwen2.5-7B improved from 23.0% to 54.6%, $\rho = 0.66$), demonstrating that iterative feedback with bounded historical memory significantly outperforms single-shot generation across model origins and datasets of varying difficulty. Three design principles distinguish our approach from prior iterative LLM optimization methods: (1) operating in an open code space rather than constrained predefined cells, (2) bounded K-step Markovian memory prevents context overflow while retaining relevant failure patterns, and (3) structured diagnostic triples explicitly model code execution failures, which alongside dual-LLM specialization (Code Generator and Prompt Improver) enables effective operation on small frozen models without fine-tuning. Our results establish that even $\leq$7B LLMs from diverse AI ecosystems can serve as effective architecture search agents when equipped with structured iterative feedback, offering a lightweight, data-efficient, and low-budget approach to reproducible neural architecture search.

References

[1] Mohamed S. Abdelfattah, Abhinav Mehrotra, Łukasz Dudziak, and Nicholas D. Lane. Zero-cost proxies for lightweight NAS. In *International Conference on Learning Representations*, 2021. 5

[2] Nada Aboudeshish, Dmitry Ignatov, and Radu Timofte. Augmentgest: Can random data cropping augmentation boost gesture recognition performance? *arXiv preprint*, arXiv:2506.07216, 2025. 5

[3] Angelica Chen, David Dohan, and Daved Hutchins. Evoprompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, 2023. 1, 3, 4, 7

[4] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint*, arXiv:2107.03374, 2021. 3

[5] Wuyang Chen, Xinyu Gong, and Zhangyang Wang. Neural architecture search on ImageNet in four GPU hours: A theoretically inspired perspective. In *International Conference on Learning Representations*, 2021. 5

[6] Saif U Din, Muhammad Ahsan Hussain, Mohsin Ikram, Dmitry Ignatov, and Radu Timofte. Ai on the edge: An automated pipeline for pytorch-to-android deployment and benchmarking. *Preprints*, 2025. 3

[7] Xuanyi Dong and Yi Yang. Nas-bench-201: Extending the scope of reproducible neural architecture search. In *International Conference on Learning Representations*, 2020. 3, 5

[8] Romain Egele, Felix Mohr, Tom Viering, and Prasanna Balaprakash. The unreasonable effectiveness of early discarding after one epoch in neural network hyperparameter optimization. *Neurocomputing*, 597:127964, 2024. 5

[9] Mohamed Gado, Towhid Taliee, Muhammad Danish Memon, Dmitry Ignatov, and Radu Timofte. Vist-gpt: Ushering in the era of visual storytelling with llms? *arXiv preprint*, arXiv:2504.19267, 2025. 3

[10] Arash Torabi Goodarzi, Roman Kochnev, Waleed Khalid, Furui Qin, Tolgay Atinc Uzun, Yashkumar Sanjaybhai Dhameliya, Yash Kanubhai Kathiriya, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. LEMUR Neural Network Dataset: Towards Seamless AutoML. *arXiv preprint*, arXiv:2504.10552, 2025. 3

[11] Jeremy Howard. Imagenette: A smaller subset of 10 easily classified classes from imagenet, 2019. 1, 5

[12] Krunal Jesani, Dmitry Ignatov, and Radu Timofte. Llm as a neural architect: Controlled generation of image captioning models under strict api contracts. *arXiv preprint*, arXiv:2512.14706, 2025. 1, 3

[13] Waleed Khalid, Dmitry Ignatov, and Radu Timofte. A retrieval-augmented generation approach to extracting algorithmic logic from neural networks. *arXiv preprint*, arXiv:2512.04329, 2025. 3

[14] Waleed Khalid, Dmitry Ignatov, and Radu Timofte. From memorization to creativity: Llm as a designer of novel neural-architectures. *arXiv preprint*, arXiv:2601.02997, 2026. 1, 3

[15] Roman Kochnev, Arash Torabi Goodarzi, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. Optuna vs

Code Llama: Are LLMs a New Paradigm for Hyperparameter Tuning? In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 5664–5674, 2025. 3

[16] Roman Kochnev, Waleed Khalid, Tolgay Atinc Uzun, Xi Zhang, Yashkumar Sanjaybhai Dhameliya, Furui Qin, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Nngpt: Rethinking automl with large language models. *arXiv preprint*, arXiv:2511.2033, 2025. 1, 3

[17] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 1, 5

[18] Guihong Li et al. Zero-shot neural architecture search: Challenges, solutions, and opportunities. *IEEE TPAMI*, 46(6): 4071–4088, 2024. 1, 3, 7

[19] Ming Lin, Pichao Wang, Zhenhong Sun, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li, and Rong Jin. Zen-NAS: A zero-shot NAS for high-performance image recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12265–12275, 2021. 5

[20] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *ICLR*, 2019. 1, 3

[21] Joseph Mellor, Jack Turner, Amos Shersby, and Elliot J. Shersby. Neural architecture search without training. In *ICML*, pages 7588–7598, 2021. 1, 3, 7

[22] Joe Mellor, Jack Turner, Amos Storkey, and Elliot J. Crowley. Neural architecture search without training. In *Proceedings of the 38th International Conference on Machine Learning*, pages 7588–7598. PMLR, 2021. 5

[23] Yash Mittal, Dmitry Ignatov, and Radu Timofte. Preparation of fractal-inspired computational architectures for advanced large language model analysis. *arXiv preprint*, arXiv:2511.07329, 2025. 1, 3

[24] Hieu Pham, Melody Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *ICML*, pages 4095–4104, 2018. 3

[25] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *AAAI*, pages 4780–4789, 2019. 1, 2, 7

[26] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625 (7995):468–475, 2023. 1, 3, 4, 7

[27] Robin Ru, Clare Lyle, Lisa Schut, Miroslav Fil, Mark van der Wilk, and Yarin Gal. Speedy performance estimation for neural architecture search. *Advances in Neural Information Processing Systems*, 34:4079–4092, 2021. 4

[28] Bhavya Rupani, Dmitry Ignatov, and Radu Timofte. Exploring the collaboration between vision models and llms for enhanced image classification. *Preprints*, 2025. 3

[29] Usha Shrestha, Dmitry Ignatov, and Radu Timofte. From brute force to semantic insight: Performance-guided data transformation design with llms. *arXiv preprint*, arXiv:2601.03808, 2026. 3

[30] Tolgay Atinc Uzun, Dmitry Ignatov, and Radu Timofte. Closed-loop llm discovery of non-standard channel priors in vision models. *arXiv preprint*, arXiv:2601.08517, 2026. 1, 3

[31] Tolgay Atincand Uzun, Waleed Khalid, Saif U Din, Sai Revanth Mulukuledu, Akashdeep Singh, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Yashkumar Rajeshbhai Lukhi, Ahsan Hussain, Krunal Jesani, Usha Shrestha, Yash Mittal, Roman Kochnev, Pritam Kadam, Mohsin Ikram, Harsh Rameshbhai Moradiya, Alice Arslanian, Dmitry Ignatov, and Radu Timofte. Lemur 2: Unlocking neural network diversity for ai. *arXiv preprint*, 2026. 3

[32] Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Enhancing llm-based neural network generation: Few-shot prompting and efficient validation for automated architecture design. *arXiv preprint*, arXiv:2512.24120, 2025. 1, 3

[33] Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*, 2023. 5

[34] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2023. 1, 3, 4, 7

[35] Fei Ye, Haoran Yang, Xiaohan Cai, et al. Reevo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, 2024. 3, 4, 7

[36] Yihang Yin, Siyu Huang, and Xiang Zhang. BM-NAS: Bilevel multimodal neural architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8901–8909, 2022. 5

[37] Chris Ying, Aaron Klein, Eric Christiansen, Esteban Real, Kevin Murphy, and Frank Hutter. Nas-bench-101: Towards reproducible neural architecture search. In *International Conference on Machine Learning*, pages 7105–7114. PMLR, 2019. 5

[38] Kaicheng Yu, Christian Sciuto, Martin Jaggi, Claudiu Musat, and Mathieu Salzmann. Evaluating the search phase of neural architecture search. In *International Conference on Learning Representations*, 2020. 5

[39] Yuhang Zhong, Yuhao Liu, Xiaotong Liu, and Tao Chen. Large language model assisted adversarial robustness neural architecture search. *IEEE Transactions on Artificial Intelligence*, 2024. 1, 3, 4

[40] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *ICLR*, 2017. 1, 2, 7